Web Application Security Testing Report

Target Application: DVWA / Pentest-Ground

Tools Used: Burp Suite and Kali Linux

---

1. Introduction

This report documents the web application security testing performed on a controlled DVWA and Pentest-Ground environment. The purpose of this testing was to identify common web application vulnerabilities and understand how insecure input handling can affect an application.

The testing focused mainly on Command Injection and Reflected Cross-Site Scripting (XSS). Burp Suite was used to inspect HTTP requests and responses, while Kali Linux was used as the testing environment.

2. Tools Used

- Kali Linux
- Burp Suite Community Edition
- DVWA
- Web Browser

3. Command Injection Testing

Command Injection occurs when an application passes user-controlled input to a system command without proper validation. This can allow unintended commands to be executed by the underlying operating system. Testing was performed only in the authorized DVWA laboratory environment.

Normal Application Test

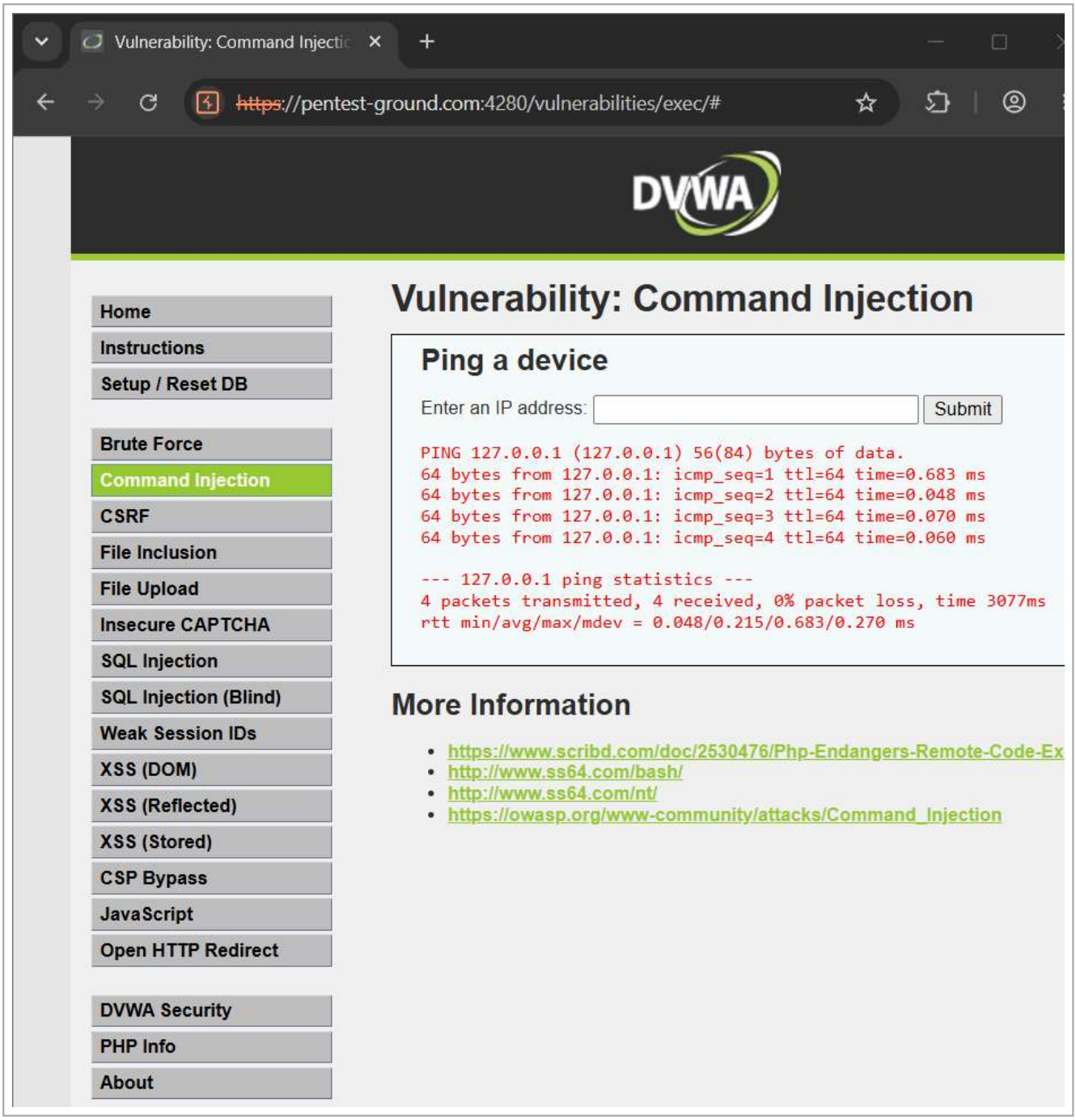

Figure 1: Command Injection Normal Test

Proof of Concept

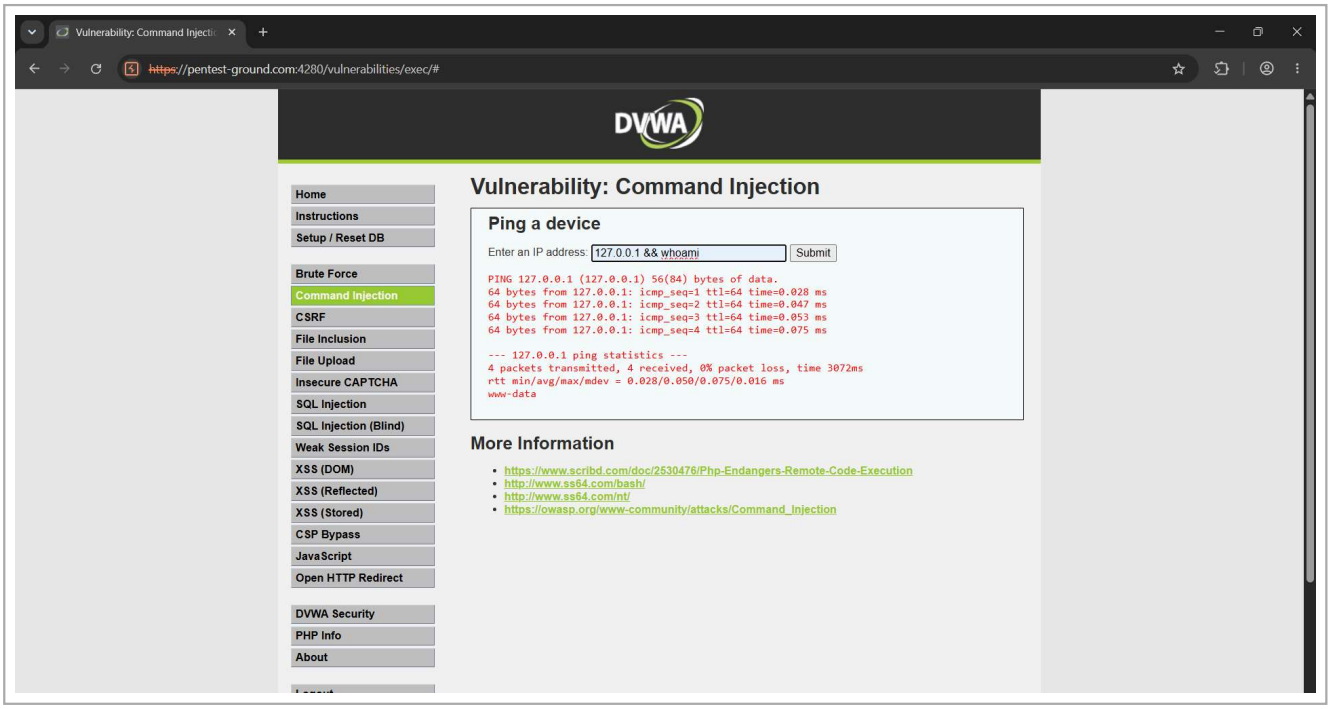

Figure 2: Command Injection Proof of Concept

The result demonstrated that the application accepted input that could influence the system command being executed. This shows the importance of validating and sanitizing user input before passing it to system commands.

4. Environment Setup

Before performing the vulnerability testing, the DVWA application was accessed and its security configuration was verified. The following screenshots show the login page, setup check, and low security configuration used for testing.

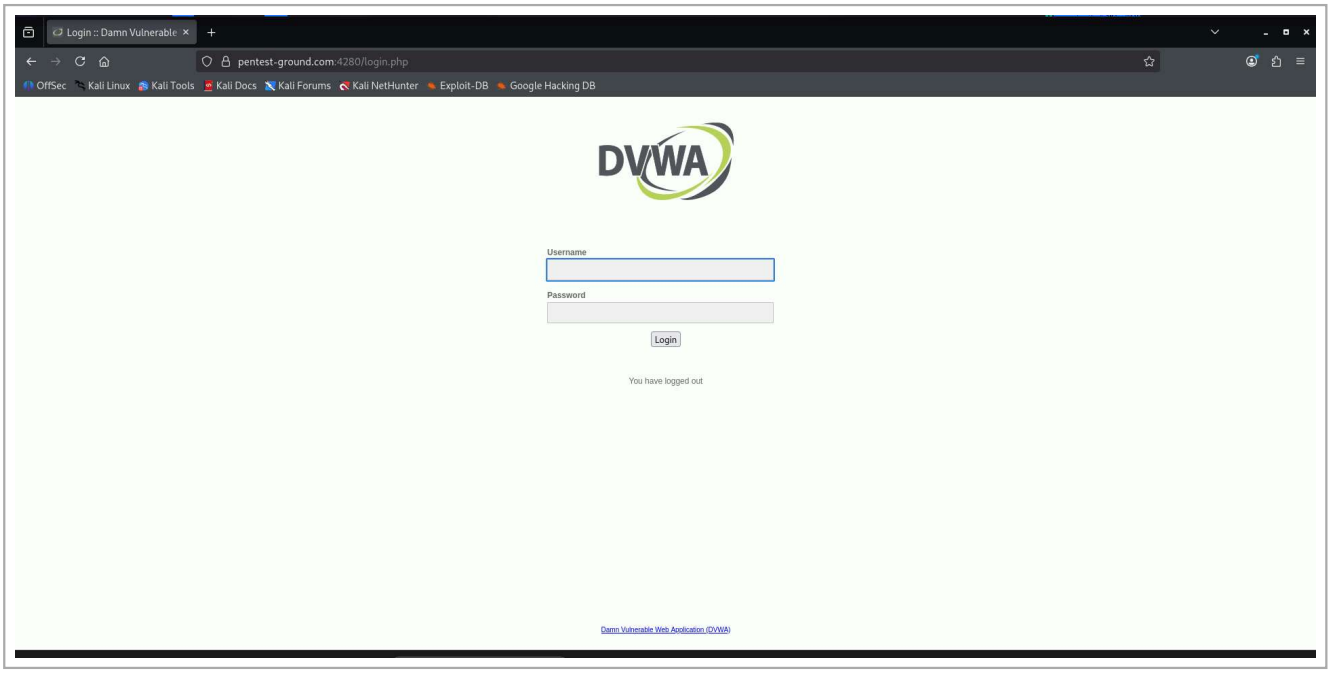

Figure 3: DVWA Login Page

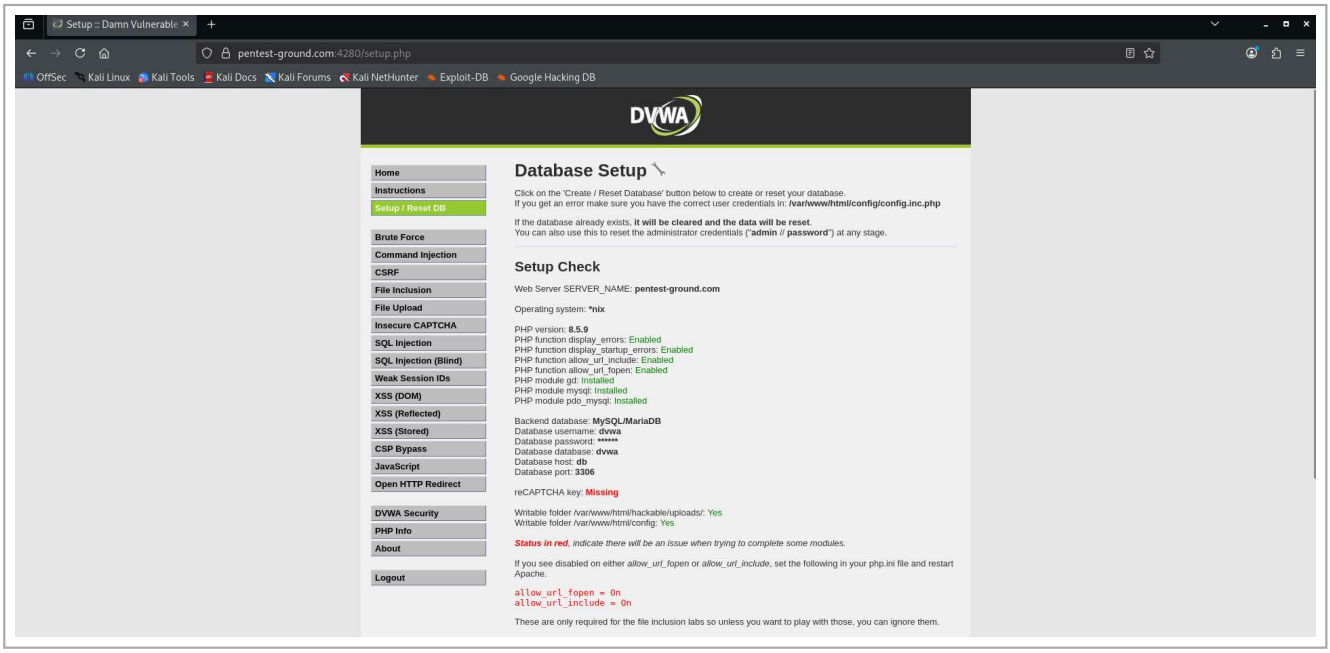

Figure 4: DVWA Setup Check

Figure 5: DVWA Security Configuration

5. Burp Suite Request Interception

Burp Suite was configured as a proxy to intercept and inspect HTTP requests sent between the browser and the target application. This helped in understanding how user input was transmitted to the server.

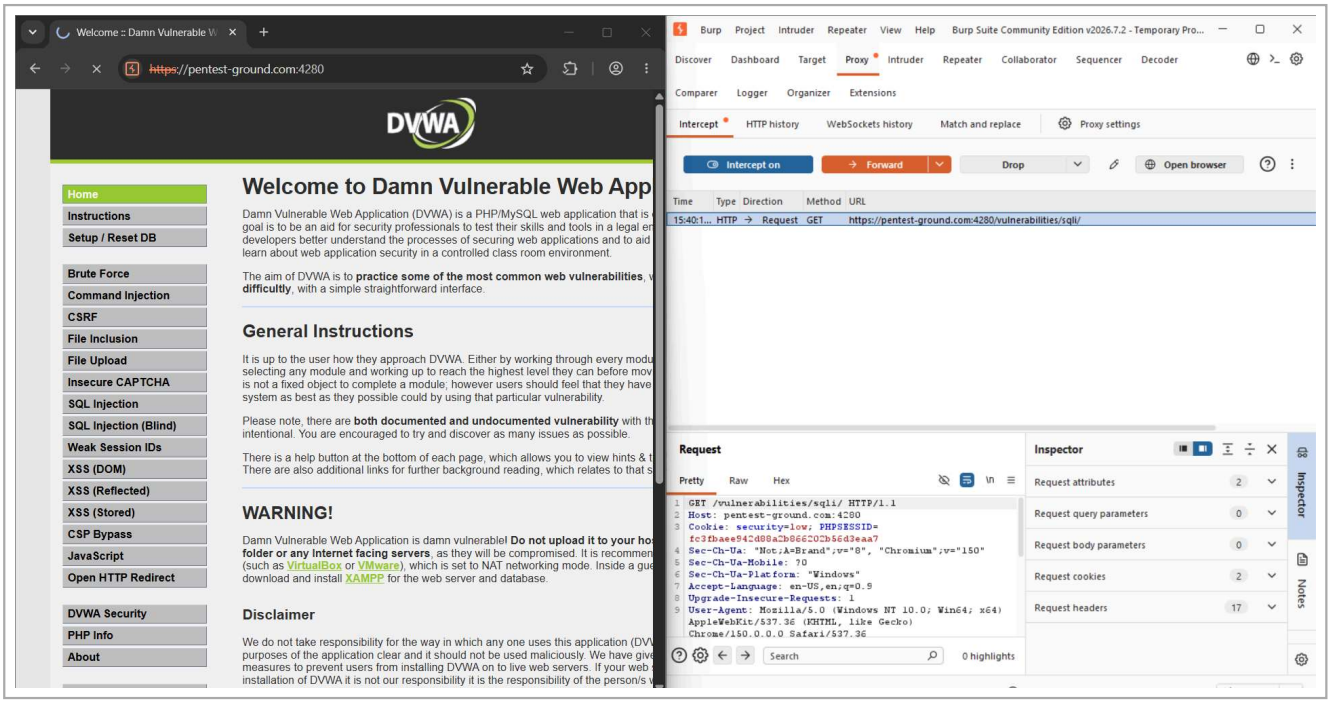

Figure 6: HTTP Request Intercepted Using Burp Suite

6. Reflected Cross-Site Scripting Testing

Reflected Cross-Site Scripting occurs when user-supplied input is immediately returned by the web application without proper output encoding or validation. This can result in client-side code being executed in the victim's browser.

Normal Application Behaviour

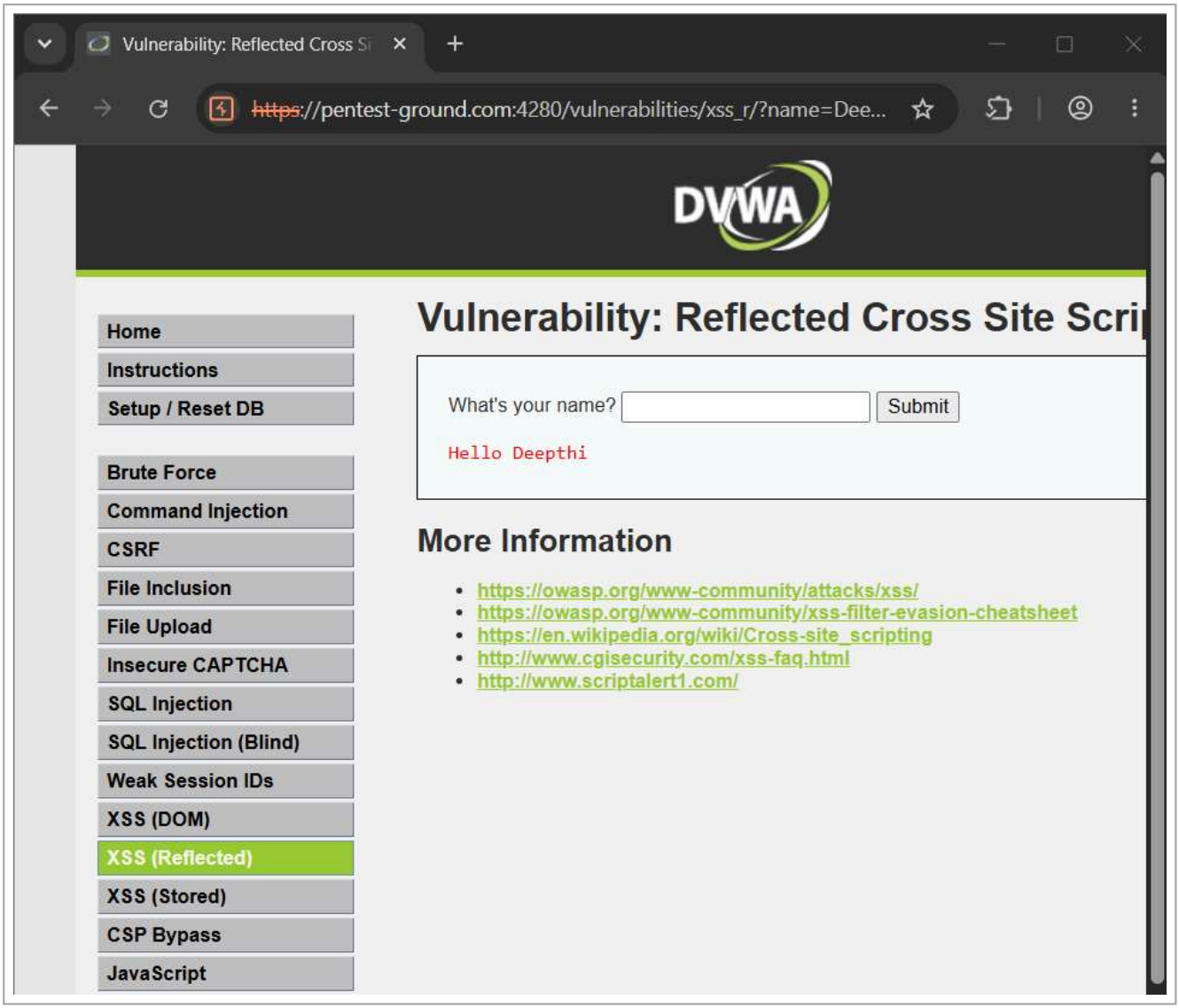

Figure 7: Normal Reflected XSS Application Behaviour

Reflected XSS Proof of Concept

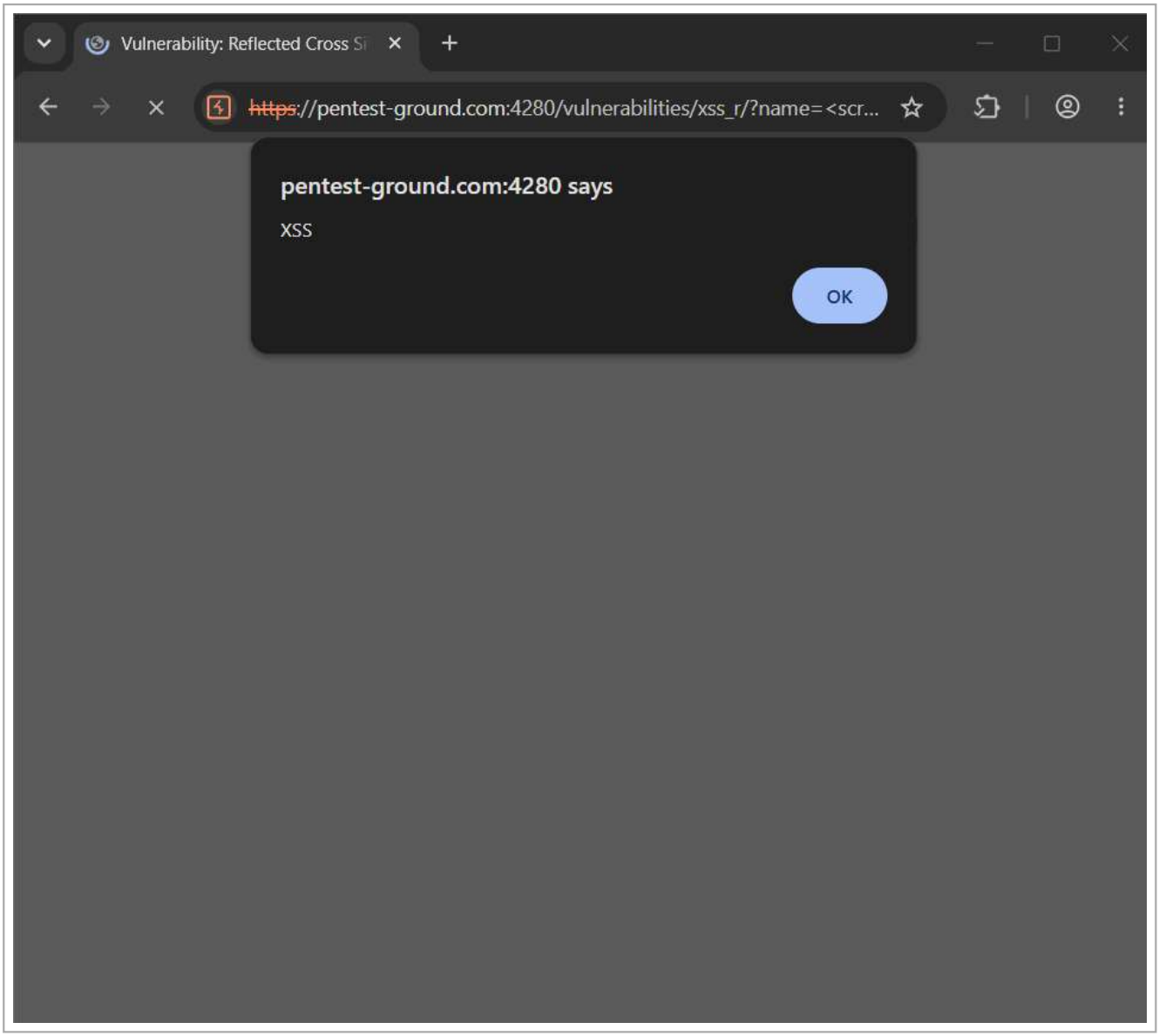

Figure 8: Reflected XSS Proof of Concept

The testing confirmed that reflected user input could be interpreted by the browser, demonstrating the presence of a Reflected XSS vulnerability in the controlled testing environment.

7. SQL Injection Testing

SQL Injection testing was attempted in the DVWA environment. However, the database configuration was not functioning correctly and the required database tables were missing. Because of this configuration issue, SQL Injection testing could not be completed successfully.

The application displayed database-related errors indicating that the required DVWA database tables did not exist. Therefore, SQL Injection was documented as an attempted test that could not be completed due to the environment configuration problem.

8. Findings Summary

> **Finding 1: Command Injection**
>
> User input was able to influence command execution in the intentionally vulnerable laboratory environment.
>
> **Finding 2: Reflected Cross-Site Scripting**
>
> User-controlled input was reflected in the application response, demonstrating the risk of client-side script execution.
>
> **Finding 3: SQL Injection**
>
> SQL Injection testing could not be completed because the DVWA database was not properly configured and required tables were missing.

9. Recommendations

- Validate all user input using allowlists where possible.
- Avoid directly passing user input to operating system commands.
- Use secure APIs instead of constructing system commands manually.
- Encode output before displaying user-controlled data in web pages.
- Use parameterized queries to prevent SQL Injection.
- Do not display detailed database or server errors to users.
- Regularly perform security testing and vulnerability assessments.

10. Conclusion

During this web application security testing, Command Injection and Reflected Cross-Site Scripting vulnerabilities were successfully demonstrated in an authorized and controlled testing environment. Burp Suite was used to inspect HTTP requests, while Kali Linux was used as the testing platform.

SQL Injection testing could not be completed because of a database configuration error. Overall, the testing demonstrated the importance of proper input validation, output encoding, secure database handling, and regular security testing of web applications.

Web Application Security Testing Report

DVWA | Burp Suite | Kali Linux

GitHub Repository:

https://github.com/Deepthi452005/NovaShyld-Task_4